Atlas AQAR: Academic Rewrite of the Project Explanation

Agentic AI-Driven Real-Estate Research and Investment Intelligence System (DEPI Round 5, Project NHA-5-072)

Abstract

Atlas AQAR is a multi-agent, retrieval-augmented system for evidence-grounded research and investment analysis in the Egyptian residential real-estate market. It is implemented as a stateful, cyclic workflow in LangGraph. Five specialized agents (Planner, Researcher, Analyzer, Writer, and Critic) decompose a user query into dependency-ordered sub-tasks, retrieve supporting evidence from a ChromaDB vector store, and synthesize a structured report. All financial quantities, including amortization schedules, net rental yields, capital appreciation, and sensitivity analyses, are computed by a deterministic Python module rather than by the language model. Each factual or quantitative claim in the output is linked to a unique Evidence ID (e.g., [E001]), which makes the report auditable. A Critic agent applies a seven-criterion rubric and may trigger bounded revision cycles before the final report is released. The system is exposed through a FastAPI service and a Streamlit dashboard, and is deployed with Docker Compose.

1. Problem Statement

Real-estate investment decisions in Egypt are constrained by four structural limitations:

1. **Data fragmentation.** Listings, price indices, and developer records are dispersed across heterogeneous and largely unstructured sources.
2. **Information asymmetry.** Buyers generally lack verifiable data on developers' historical delivery performance and financial solvency.
3. **Opacity of payment structures.** Installment plans frequently contain deferred maintenance deposits and back-loaded balloon payments, which make true cost difficult to assess.
4. **Unreliability of general-purpose language models.** Unconstrained LLMs are prone to fabricating entities and prices and to making arithmetic errors, so they are unsuitable as sole sources for financial analysis.

Atlas AQAR addresses these limitations by combining semantic retrieval, deterministic computation, and agent-level verification within a single pipeline.

2. Objectives

ID	Objective	Operationalization
O1	Task decomposition	Partition each query into 3–6 dependency-ordered sub-tasks
O2	Computational determinism	Perform all financial arithmetic in non-LLM code
O3	Evidential traceability	Require an Evidence ID for every quantitative or factual claim
O4	Self-critique	Apply a seven-criterion rubric with bounded revision
O5	Provider independence	Support OpenAI, Anthropic, Google Gemini, and an offline mock provider
O6	Modular deployment	Deliver a FastAPI backend, Streamlit interface, and containerized stack within four weeks

3. Methodology

Architecture. Orchestration uses a cyclic state graph in which a typed `GraphState` (Pydantic v2) is passed between agent nodes. Conditional edges route control according to the Critic's verdict. Termination is guaranteed by two bounds: at most 10 workflow iterations per request and at most 3 Critic-initiated revisions.

Retrieval. A curated corpus of property listings, developer profiles, and market reports is embedded and indexed in ChromaDB. The Researcher agent follows a ReAct pattern (thought, action, observation), issuing tool calls until sufficient evidence has accumulated.

Quantitative layer. Amortization, balloon-payment modeling, ROI, and scenario sensitivity are implemented as pure functions and exposed to the agents as tools. Arithmetic is excluded from LLM text generation by design.

Verification. The Critic evaluates each draft for factual grounding, mathematical accuracy, and completeness. Drafts that fail are returned with targeted revision directives.

Execution. The work was organized into four one-week sprints across six functional roles: architecture, agent reasoning, data and retrieval, quantitative methods, backend, and interface/QA/DevOps.

4. Evaluation

The system was assessed on five benchmark scenarios against pre-specified thresholds:

Dimension	Metric	Threshold	Observed
Task completion	Sub-task execution rate	⩾ 80%	100%
Fact verification	Citation precision	> 90%	100%
Financial reliability	Agreement with reference computation	100%	100%
Editorial quality	Critic pass rate	⩾ 85%	100%
Hallucination control	Unsupported-claim rate	⩽ 2%	0%
Efficiency	Offline benchmark runtime	< 5 s	0.12 s

5. Scope and Limitations

The system does not perform real-time scraping of unlicensed listing portals, user authentication or billing, native mobile deployment, integration with banking or mortgage-approval services, or legal advisory functions. Report generation in Arabic is deferred to a later version. The corpus is a curated sample covering New Cairo, the New Administrative Capital, and Sheikh Zayed, so findings should not be generalized beyond these segments.

Suggested Qualifications for Rigor

An academic reader would likely challenge the evaluation section as currently framed:

- Uniform scores of 100% and 0% on only five scenarios do not support strong generalization. State the sample size explicitly and avoid language such as "completely eliminated" or "zero hallucination" as general claims. "No unsupported claims were observed in the five evaluated scenarios" is defensible.

- A 0.12 s runtime across five full workflows suggests the mock LLM was used. Say so, since latency and citation behavior with live models will differ.
- "Financial determinism" is guaranteed by construction, but *correct* financial output still depends on correct inputs and on the LLM passing the right parameters to the tools. Distinguish computational correctness from end-to-end correctness.
- Citation precision measures whether claims are *cited*, not whether the cited evidence actually *supports* them. Define the metric if you keep it.

Appendix A. Technology Stack

Tier	Technology	Version	Responsibility
Agentic orchestration	LangGraph, LangChain-Core	1.2.14 / 1.6.9	Stateful cyclic graph execution, agent nodes, conditional edge routing
Data contracts	Pydantic v2	2.14.0	Strict typed validation of GraphState, EvidenceArsenal, and analytical payloads
Retrieval and embeddings	ChromaDB, HuggingFace	1.5.9 / 2.2.0	Persistent semantic indexing and dense similarity search
Financial analytics	Pure Python	3.11+	Amortization, net yield, capital appreciation, sensitivity grid
Backend	FastAPI, Uvicorn	0.143.0 / 0.54.0	Asynchronous REST service, health diagnostics, OpenAPI specification
Dashboard	Streamlit, Plotly	1.65.0 / 7.1.0	Report rendering and interactive financial visualization
Quality assurance	pytest, pytest-asyncio	9.1.1 / 1.4.0	Unit, integration, and benchmark assertions
Infrastructure	Docker, Docker Compose	Latest	Multi-service containerization and volume persistence

Appendix B. System Components

Agents. The Planner decomposes the query into 3–6 ordered sub-tasks. The Researcher gathers evidence through a ReAct loop over an eight-tool registry. The Analyzer integrates retrieved facts with deterministic financial outputs. The Writer produces a nine-section structured report with inline Evidence IDs. The Critic applies the seven-criterion rubric and either approves the draft or returns revision directives.

Tool registry. Eight domain tools support the agents, among them RAGRetrieverTool, PropertySearchTool, PropertyDetailsTool, DeveloperLookupTool, InstallmentCalcTool, ROIEstimatorTool, and a WebSearchTool fallback with HTML sanitization.

API surface. Versioned endpoints under `/api/v1/` include `POST /research` (workflow execution), `GET /properties` (multi-filter query), `POST /financial` (standalone calculations), and `GET /health` (probe), with interactive documentation at `/docs`. Middleware provides JSON error envelopes, rate limiting, and query timeout guards.

Dashboard. Six analytical views: Research Console, Verified Report Viewer, Property Grid, Financial Calculator, Evidence Inspector, and Observability Monitor. Visualizations include scatter plots, amortization bars, and an ROI gauge.

Appendix C. Project Timeline and Deliverables

Sprint	Days	Milestone	Principal outcomes
1	1–7	M1: Environment and data foundation	Repository, Pydantic schemas, ChromaDB ingestion, benchmark harness
2	8–14	M2: Agent suite and financial engine	Five agent nodes, eight-tool registry, deterministic engine with full unit coverage
3	15–21	M3: API and dashboard	FastAPI service, six-panel Streamlit interface, integration tests
4	22–28	M4: Containerization and release	Docker Compose validation, benchmark execution, documentation, release tag

Twelve deliverables (D01–D12) were defined, spanning the repository, data models, vector store, agent workflow, tool suite, financial engine, API, dashboard, deployment, evaluation report, project plan, and final submission package. All are recorded as completed in the source plan.

Appendix D. Division of Responsibility

Role	Primary domain	Representative contributions
Engineer 1: AI architect	Graph orchestration	StateGraph topology, iteration caps, provider adapter, OpenTelemetry tracing
Engineer 2: LLM specialist	Prompting and verification	Planner prompts, ReAct Researcher, Writer synthesis, Critic rubric, citation evaluation
Engineer 3: Data specialist	Corpus and retrieval	Property and developer datasets, ingestion pipeline, retrieval tools, SHA-256 dataset snapshots
Engineer 4: Financial specialist	Quantitative methods	Installment, ROI, and scenario modules, currency formatters, boundary testing
Engineer 5: Backend specialist	Service layer	FastAPI routes, schemas, middleware, concurrency stress test
Engineer 6: Interface and DevOps	Delivery	Repository setup, Streamlit panels, test suites, Docker Compose, release

The responsibility matrix assigns exactly one accountable engineer to each deliverable, with other engineers designated as responsible, consulted, or informed.

Appendix E. Stakeholders and Governance

Internal stakeholders comprise the six-engineer core, the project lead, the AI architect, and the QA and DevOps leads. External stakeholders include DEPI program leadership and mentors, the National Housing Authority, prospective real-estate investors, property developers, academic reviewers, and LLM infrastructure providers. Communication follows a fixed cadence: daily stand-ups through GitHub Issues, weekly formal briefings to

DEPI leadership, bi-weekly live demonstrations for institutional partners, and per-milestone documentation for reviewers. Every pull request must pass automated unit tests and linting before merge, and any architectural deviation from the original proposal requires prior review.

Appendix F. Risk Register

ID	Risk	Likelihood	Impact	Mitigation
R01	LLM API quota exhaustion or rate limiting	Medium	High	Offline mock provider, 100k session token budget, multiple provider keys
R02	Breaking changes in LangGraph or ChromaDB	Low	High	Exact version pinning, isolated virtual environments
R03	Vector store corruption or re-indexing lag	Low	Medium	Idempotent ingestion, mounted volumes, startup health probe
R04	Edge-case financial discrepancies (e.g., balloon payments)	Medium	High	Full unit coverage, strict numeric validation, math isolated from prompts
R05	Integration delays between workstreams	Medium	Medium	Typed interfaces defined in Week 1, mock responses for parallel development
R06	Windows versus Linux environment differences	Medium	Low	Cross-platform Docker builds, `.env.example` with portable paths
R07	Streamlit state loss or re-rendering latency	Low	Low	Session-state caching, paginated tables, lazy-loaded charts

Appendix G. Assumptions and Constraints

Assumptions. All engineers work with Python 3.11+ and local Docker Desktop. The curated datasets are representative of the three target market segments. At least one live provider key is available for validation, with offline mode available for local testing.

Constraints. No more than 10 workflow iterations and 3 Critic revisions per request. No arithmetic is delegated to LLM text generation. The system must run locally without mandatory paid cloud services. The four-week timeline is fixed, with weekly sprint reviews.

Appendix H. Glossary

Term	Definition
Agentic workflow	A design in which an LLM iteratively coordinates reasoning, tool use, observation, and state updates rather than answering in a single prompt
GraphState	The central typed Pydantic model passed between agents, holding the query, sub-tasks, evidence records, drafts, and critique notes
Evidence Arsenal	The collection of verified facts, listings, and computed figures gathered during research, each carrying an Evidence ID
Evidence ID	A deterministic citation tag (e.g., [E001]) assigned to a data item and referenced inline in the report
ReAct loop	A reasoning-and-acting pattern in which the agent alternates thoughts, tool actions, and observations until evidence suffices
Critic agent	An evaluator that scores drafts against a seven-criterion rubric and approves them or requests revision
DEPI	Digital Egypt Pioneers Initiative, the national program under which the project was developed